# Stack Memory vs Heap Memory in Java

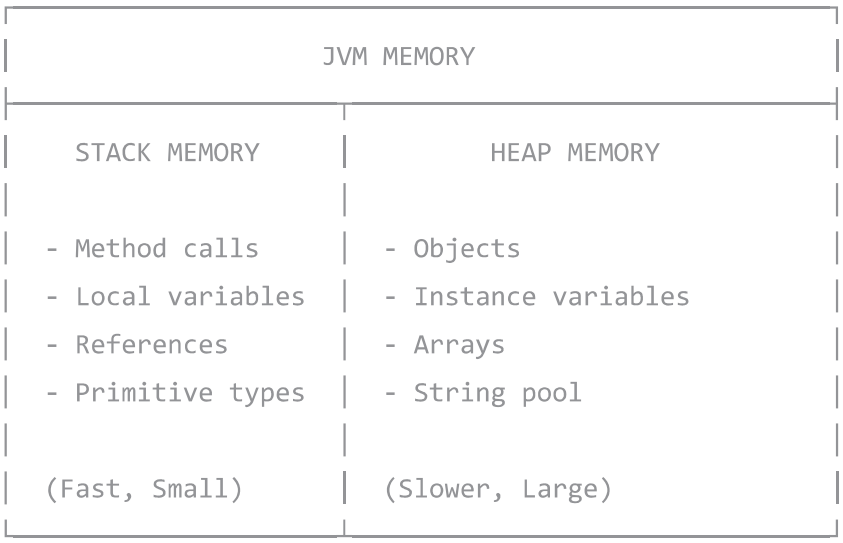
### Complete Guide with Examples and Diagrams

---

## 1. What is Memory in Java?

When a Java program runs, the JVM (Java Virtual Machine) divides memory into different parts:

| Memory Type | Purpose |
|---|---|
| **Stack Memory** | Method calls, local variables, references |
| **Heap Memory** | Objects, instance variables, arrays |




---

# 2. Stack Memory

## What is Stack Memory?

Stack memory stores:

- **Method execution frames** (each method call creates a frame)
- **Local variables** (declared inside methods)
- **Reference variables** (addresses pointing to objects)
- **Primitive data types** (int, double, boolean, etc.)

## Characteristics

| Feature | Description |
|---|---|
| **Speed** | Very fast access |
| **Size** | Small (limited) |
| **Lifetime** | Variables die when method ends |
| **Structure** | LIFO (Last In, First Out) |
| **Thread Safety** | Each thread has its own stack |
| **Management** | Automatic (no GC needed) |

## How Stack Works (LIFO)

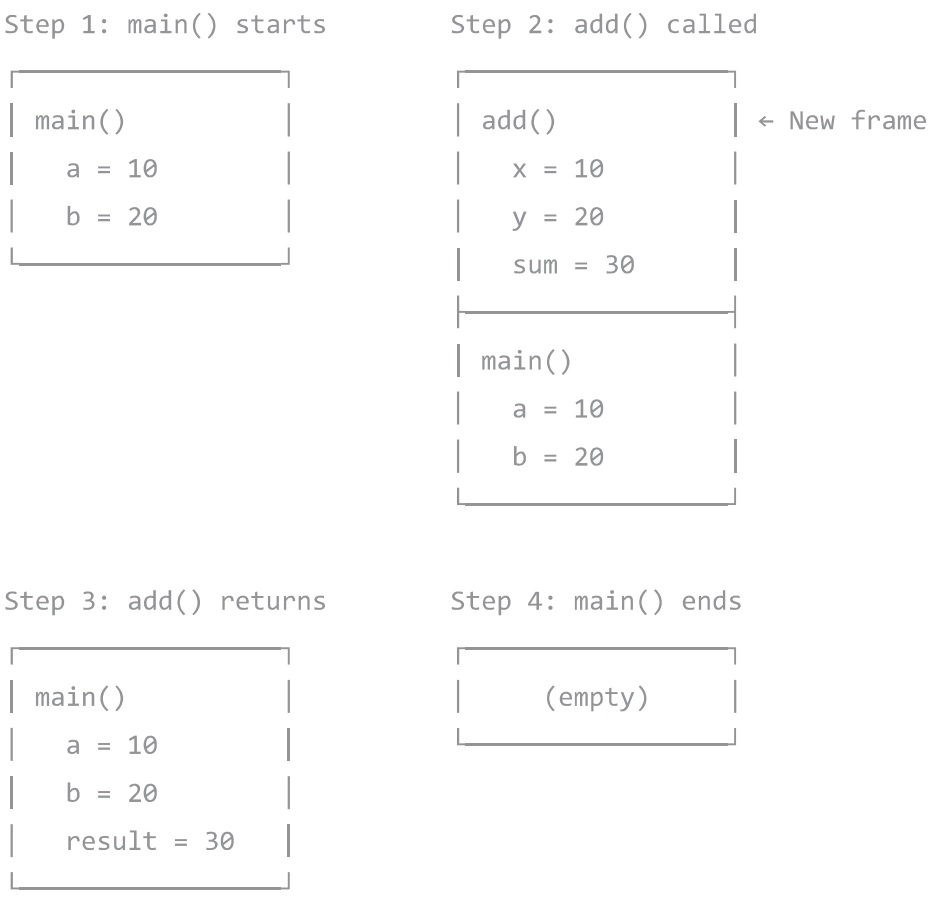
LIFO = Last In, First Out (like a stack of plates)

```
┌──────────┐
│ method3  │  ← Last in, First out
├──────────┤
│ method2  │
├──────────┤
│ method1  │
├──────────┤
│   main   │  ← First in, Last out
└──────────┘
```

## Stack Example

```
public class StackDemo {
    public static void main(String[] args) {
        int a = 10;              // 'a' in stack
        int b = 20;              // 'b' in stack
        int result = add(a, b);  // new frame created
        System.out.println(result);
    }

    static int add(int x, int y) {
        int sum = x + y;         // 'x', 'y', 'sum' in stack
        return sum;
    }
}
```

### Stack visualization:




---

# 3. Heap Memory

## What is Heap Memory?

Heap memory stores:

- **Objects** (created with `new` keyword)
- **Instance variables** (variables inside objects)
- **Arrays**
- **String pool**

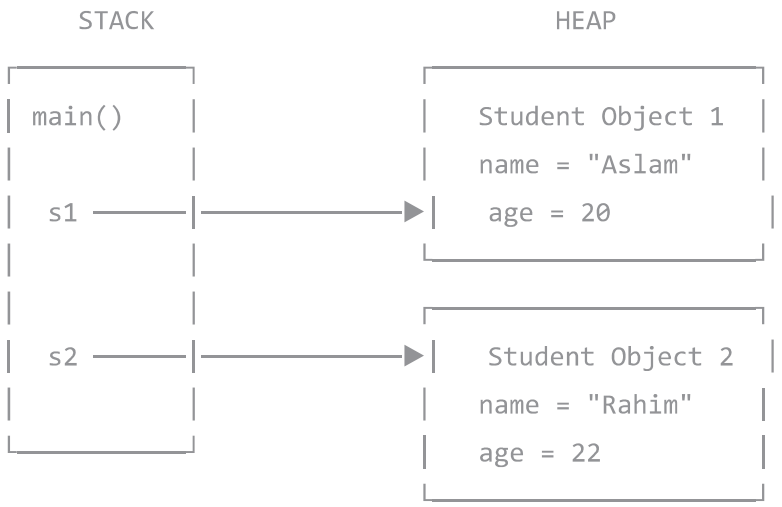
## Characteristics

| Feature | Description |
|---|---|
| **Speed** | Slower than stack |
| **Size** | Large (can grow) |
| **Lifetime** | Until garbage collected |
| **Structure** | No specific order |
| **Thread Safety** | Shared among all threads |
| **Management** | Garbage Collector |

## Heap Example

```
class Student {
    String name;    // In heap (instance variable)
    int age;        // In heap (instance variable)

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

public class HeapDemo {
    public static void main(String[] args) {
        Student s1 = new Student("Aslam", 20);
        Student s2 = new Student("Rahim", 22);
    }
}
```

### Memory visualization:




---

# 4. Stack vs Heap — Comparison

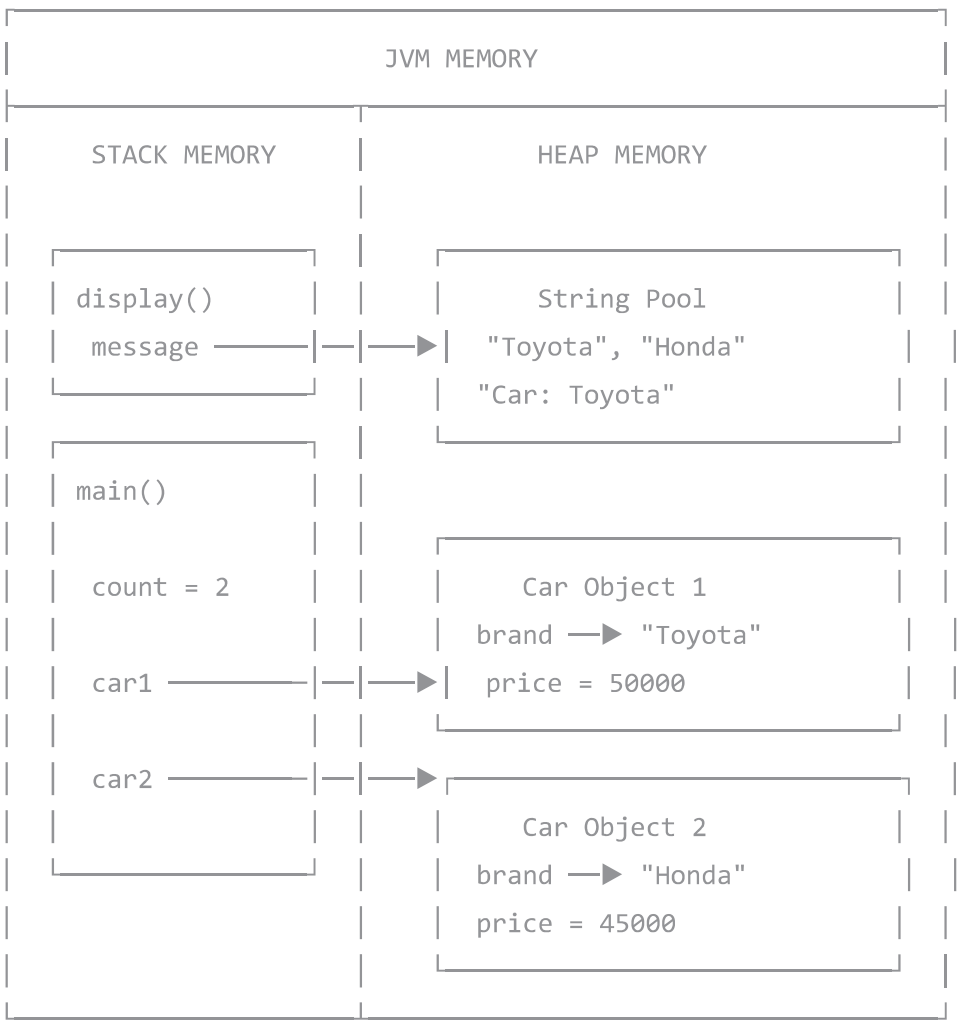
| Feature | Stack Memory | Heap Memory |
|---|---|---|
| **Stores** | Primitives, references | Objects, arrays |
| **Speed** | Very fast | Slower |
| **Size** | Small (few MB) | Large (can be GB) |
| **Lifetime** | Until method ends | Until GC collects |
| **Access** | Only by owning thread | Shared by all threads |
| **Allocation** | Contiguous | Random |
| **Management** | Automatic (LIFO) | Garbage Collector |
| **Error** | StackOverflowError | OutOfMemoryError |

---

# 5. Complete Memory Example

```
class Car {
    String brand;      // Heap
    int price;         // Heap

    Car(String brand, int price) {
        this.brand = brand;
        this.price = price;
    }

    void display() {
        String message = "Car: " + brand;  // Stack
        System.out.println(message);
    }
}

public class MemoryDemo {
    public static void main(String[] args) {
        int count = 2;                          // Stack
        Car car1 = new Car("Toyota", 50000);    // Ref: Stack, Obj: Heap
        Car car2 = new Car("Honda", 45000);     // Ref: Stack, Obj: Heap

        car1.display();
    }
}
```

# Memory Diagram



# 6. What Goes Where?

## Stored in STACK

```
public void example() {
    int a = 10;              // Primitive ✓
    double b = 3.14;         // Primitive ✓
    boolean c = true;        // Primitive ✓

    Student s1;              // Reference only ✓
    String name;             // Reference only ✓
    int[] arr;               // Reference only ✓
}
```

## Stored in HEAP

```
public void example() {
    Student s1 = new Student();     // Object ✓
    String name = new String("Hi"); // Object ✓
    int[] arr = new int[5];         // Array ✓

    // Instance variables inside objects ✓
}
```

### Quick Reference

```
┌─────────────────────────────────────────────────┐
│              WHAT GOES WHERE?                   │
├──────────────────────────┬──────────────────────┤
│         STACK            │         HEAP         │
├──────────────────────────┼──────────────────────┤
│ ✓ Primitive variables    │ ✓ Objects (new)      │
│ ✓ Reference variables    │ ✓ Instance variables │
│ ✓ Method parameters      │ ✓ Arrays             │
│ ✓ Local variables        │ ✓ String objects     │
│ ✓ Return addresses       │ ✓ Static variables   │
└──────────────────────────┴──────────────────────┘
```

---

# 7. Memory Errors

## StackOverflowError

Caused by infinite recursion:

```
public class StackOverflowDemo {
    public static void main(String[] args) {
        recursiveMethod();  // ❌ StackOverflowError
    }

    static void recursiveMethod() {
        recursiveMethod();  // Calls itself forever
    }
}
```

```
Stack fills up:
┌──────────────────────┐
│ recursiveMethod()    │
├──────────────────────┤
│ recursiveMethod()    │
├──────────────────────┤
│ recursiveMethod()    │
├──────────────────────┤
│       ...            │ ← Stack full! ERROR
└──────────────────────┘
```

### OutOfMemoryError

Caused by too many objects:

```
public class OutOfMemoryDemo {
    public static void main(String[] args) {
        List<int[]> list = new ArrayList<>();

        while (true) {
            list.add(new int[1000000]);
            // ❌ OutOfMemoryError
        }
    }
}
```

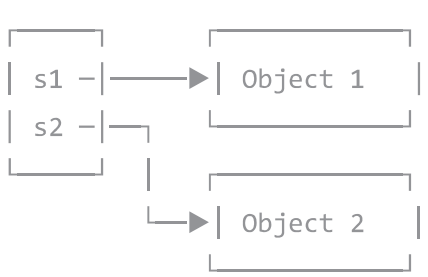
---

## 8. Garbage Collection

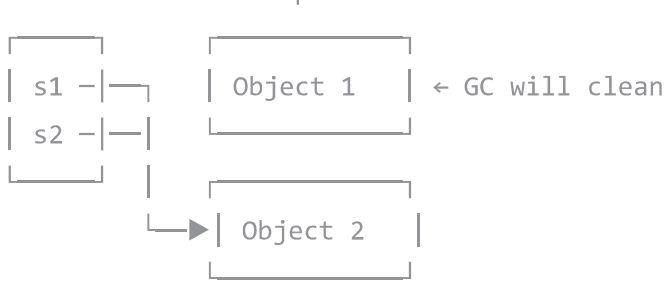
Objects in heap are cleaned when no reference points to them:

```
public class GarbageDemo {
    public static void main(String[] args) {
        Student s1 = new Student("Aslam");  // Object 1
        Student s2 = new Student("Rahim");  // Object 2

        s1 = s2;    // Object 1 has no reference
                    // Object 1 eligible for GC

        s1 = null;
        s2 = null;  // Both objects eligible for GC
    }
}
```

Step 1: Both have references




Step 2: s1 = s2 (Object 1 orphaned)




---

# 9. Practical Example

```
class Employee {
    String name;           // Heap
    int salary;            // Heap
    static String company = "TechCorp";  // Method Area

    Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }

    void giveBonus(int bonus) {
        int newSalary = salary + bonus;  // Stack
        this.salary = newSalary;         // Updates Heap
    }
}

public class MemoryExample {
    public static void main(String[] args) {
        int bonus = 5000;                      // Stack
        Employee e1 = new Employee("Aslam", 50000);  // Heap
        Employee e2 = new Employee("Rahim", 45000);  // Heap

        e1.giveBonus(bonus);
        e2.giveBonus(bonus);
    }
}
```

# 10. Summary Table

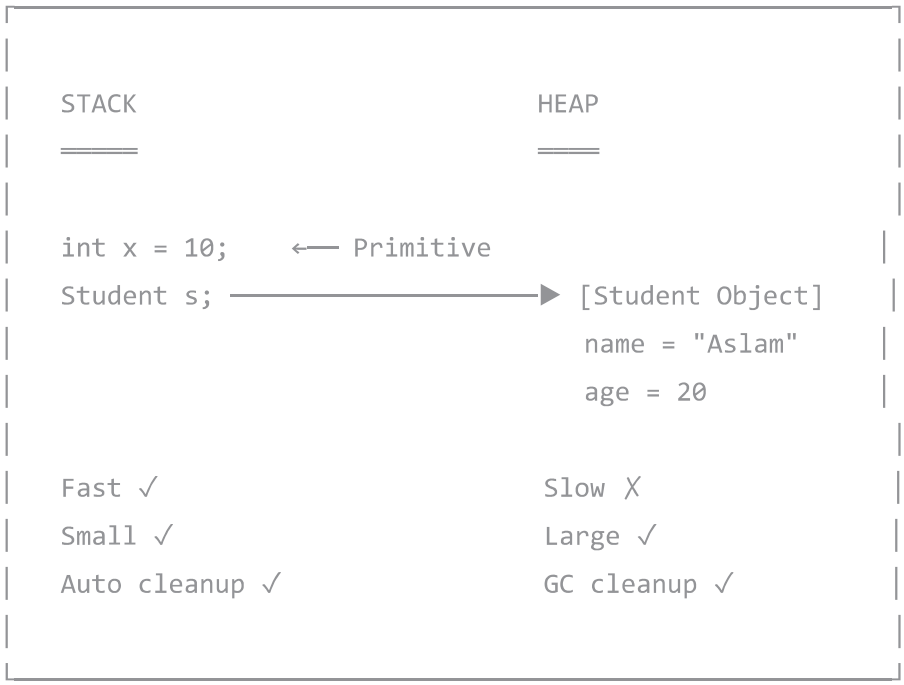
| Question | Stack | Heap |
|---|---|---|
| **What stored?** | Primitives, references | Objects, arrays |
| **Created by?** | Method call | `new` keyword |
| **Lifetime?** | Method execution | Until GC |
| **Speed?** | Very fast | Slower |
| **Size?** | Small (MBs) | Large (GBs) |
| **Access?** | Owner thread only | All threads |
| **Order?** | LIFO | No order |
| **Error?** | StackOverflowError | OutOfMemoryError |
| **Managed by?** | JVM automatic | Garbage Collector |

---

# Quick Memory Rules

```
┌──────────────────────────────────────────────────┐
│           QUICK MEMORY RULES                     │
├──────────────────────────────────────────────────┤
│                                                  │
│  1. PRIMITIVES (int, double, etc.)               │
│     → Always in STACK                            │
│                                                  │
│  2. REFERENCE VARIABLES (Student s1)             │
│     → Variable in STACK                          │
│     → Points to object in HEAP                   │
│                                                  │
│  3. OBJECTS (new Student())                      │
│     → Always in HEAP                             │
│                                                  │
│  4. INSTANCE VARIABLES (inside object)           │
│     → In HEAP (part of object)                   │
│                                                  │
│  5. LOCAL VARIABLES (inside method)              │
│     → In STACK                                   │
│                                                  │
│  6. STATIC VARIABLES                             │
│     → In Method Area (part of Heap)              │
│                                                  │
└──────────────────────────────────────────────────┘
```

# Visual Summary




---

*Created for: Md. Aslam Howlader*

*Date: February 2026*

'; triggerDownload(docTitle.replace(/[^a-zA-Z0-9\s-]/g, '') + '.html', html, 'text/html;charset=utf-8'); } function downloadPDF() { document.getElementById('downloadDropdown').classList.remove('show'); // Force light mode for PDF (dark backgrounds print poorly) const wasDark = document.documentElement.classList.contains('dark'); if (wasDark) document.documentElement.classList.remove('dark'); const element = document.getElementById('document-content'); const opt = { margin: [0.75, 0.75, 0.75, 0.75], filename: docTitle.replace(/[^a-zA-Z0-9\s-]/g, '') + '.pdf', image: { type: 'jpeg', quality: 0.98 }, html2canvas: { scale: 1.5, useCORS: true, backgroundColor: '#ffffff' }, jsPDF: { unit: 'in', format: 'letter', orientation: 'portrait' }, pagebreak: { mode: ['css', 'legacy'] } }; html2pdf().set(opt).from(element).save().then(function() { if (wasDark) document.documentElement.classList.add('dark'); }); } /* Syntax highlighting and mermaid rendering */ (async function init() { const isDark = window.matchMedia('(prefers-color-scheme: dark)').matches; // Highlight code blocks (skip mermaid) document.querySelectorAll('pre code').forEach(function(block) { const lang = block.className.replace('language-', ''); if (lang !== 'mermaid') { hljs.highlightElement(block); } }); updateHljsTheme(isDark); // Render mermaid diagrams try { mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); const

```
mermaidBlocks = document.querySelectorAll('pre code.language-mermaid'); for (let i = 0; i < mermaidBlocks.length; i++) { const block = mermaidBlocks[i]; const pre = block.parentElement; const code = block.textContent; try { const { svg } = await mermaid.render('mermaid-' + i, code); const container = document.createElement('div'); container.className = 'mermaid-container'; container.innerHTML = svg; pre.replaceWith(container); } catch (err) { console.warn('Mermaid render failed:', err); } } } catch (err) { console.warn('Mermaid init failed:', err); } })(); function updateHljsTheme(isDark) { document.getElementById('hljs-dark').disabled = !isDark; document.getElementById('hljs-light').disabled = isDark; } /* Dark mode sync from parent */ window.addEventListener('message', function(e) { if (e.data && e.data.type === 'darkMode') { const isDark = e.data.dark; document.documentElement.classList.toggle('dark', isDark); updateHljsTheme(isDark); // Re-init mermaid theme try { mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); } catch (err) {} } });
```